

Chapitre 3

Tutoriel

- Parlons d'un des grands classiques de la programmation : la **récursivité**. Vous devriez maintenant être familier avec la chronologie d'exécution de votre code. Lorsque votre programme est exécuté, il commence avec la fonction `main`.
- Si votre fonction `main` appelle une autre fonction `F`, la fonction `main` est suspendue. Les paramètres de `F` sont calculés et mis à disposition dans le code de `F`. Lorsque `F` se termine, elle retourne une valeur, et la fonction `main` suspendue continue.
- Que se passe-t-il si `F` appelle également une autre fonction `F`? À tout moment, une seule fonction est exécutée à la fois, et vous avez une chaîne de fonctions suspendues, commençant par `main`.



Prenez une pause ici et assurez-vous de comprendre ce comportement d'exécution avant de continuer ! Validez votre compréhension avec l'un de vos pairs.

- Maintenant, ajoutons une torsion : que se passe-t-il si votre fonction `F` appelle... la fonction `F` elle-même ? C'est tout à fait possible en C, et cela s'appelle la **récursivité**. Selon notre chronologie d'exécution, cela signifie que `F` est suspendue, ses variables existent toujours en RAM, et les variables locales de `F` - la seconde - sont créées.
- Le point clé est de comprendre que l'ordinateur agit comme si les première et seconde fonctions `F` étaient complètement différentes. Les variables de la première fonction `F` seront différentes en mémoire des variables de la seconde fonction `F`.
- Créez un répertoire de travail et écrivez une fonction récursive simple qui s'appelle elle-même depuis `main` :

main.c

```
void f()
{
    f();
}
```

Compilez et exécutez ce programme. Que se passe-t-il ? Votre programme devrait planter très rapidement.



Nous ne pouvons pas avoir une chaîne infinie de fonctions suspendues, car chacune demande de la mémoire et la mémoire est finie.

- Ajoutez un compteur pour voir combien de fois la fonction est appelée avant de planter :

main.c

```
void f()
{
    static int count = 0;
    count++;
    printf("%d\n", count);
    f();
}
```

- Exécutez à nouveau et notez le dernier nombre imprimé. C'est à peu près le nombre d'appels récursifs que votre système peut gérer.
- Maintenant, empêchez le plantage en arrêtant la récursion avant la limite. Utilisez votre compteur pour arrêter d'appeler la fonction à nouveau :

main.c

```
void f()
{
    static int count = 0;
    count++;
    if (count < 1000) // Utilisez un nombre inférieur à votre
        limite de plantage
        f();
}
```

- Votre programme devrait maintenant se terminer normalement. Testez-le.
- Supprimez l'impression à l'intérieur de votre fonction. En utilisant un seul `printf` dans `main`, montrez le nombre d'appels récursifs en utilisant la valeur de retour :

main.c

```
int f()
{
    static int count = 0;
    count++;
    if (count < 1000)
        return (1 + f());
    return (1);
}
```

- La fonction doit calculer le total à travers les valeurs de retour des appels récursifs, et non simplement retourner un nombre fixe.
- Chaque appel récursif a son propre ensemble de paramètres et de variables locales. Ils n'interfèrent pas les uns avec les autres même s'ils sont dans la même fonction.